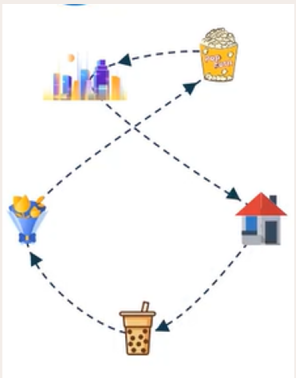
数据分析方法——优化模型求解

优化模型是数学建模里面用途最多的模型，也是最重要的

一般来说，优化模型可以简单理解为 **求极值、求最值**

女朋友跟你说，周五的约会想去喝奶茶、买花、看电影、看夜景。你自信打开导航:"按顺序走不就完了?"

周五晚上，你策划了完美约会先到网红奶茶店打卡(A点)，再带她去买花(B点)，接着转场电影院(C点)，登上城市夜景天台(D点)，最后送她回家(E点)。



结果是计划赶不上变化

1)去奶茶店的时候没有买花，导致没有道具拍照;

2)先买了花导致进电影院时花没有地方存放;

3)看完电影后地标建筑已经关灯了，导致在城市夜景天台没看到好看的夜景

4)按照这个顺序走绕路了。

那么怎么样才能避免这些情况呢?

考虑的目标

1、体验价值最大化

2、时间效率优化

3、成本与资源控制

约束条件

约束1:关键场景(如拍照点)所需道具必须在抵达前准备好;

约束2: 需考虑后续场所对携带物品的限制;

约束3:时间敏感型景点(如灯光秀)必须在有效时间段内访问;

约束4:节点顺序需符合地理逻辑。

是否有突发事件发生的可能?

发生后如何重新安排行程?

讲了这么多，那么什么是规划模型呢?

在给定的限制条件下，找到一个最佳的解决方案。简单来说，就是 **求极值，求最值**

这里的"最佳"指的是让某个特定的目标达到最大化(比如 利润、效率、速度)或最小化(比如成本、时间、风险)。

应用场景:

商业、制造:提高效率、降低成本、增加利润。

工程:设计出性能更好、更安全的系统和产品(工程)。

供应链、能源:更有效地分配资源(供应链、能源)。

物流、导航:找到最佳路线(物流、导航)。

计算机:训练更准确的机器学习模型。

金融:做出更优的投资决策(金融)。

规划模型的三个组成部分

优化目标

目标函数(Objective Function)是 优化模型中需要被最大化或最小化的数学表达式。它反映了决策者希望优化的目标。

例如

最大化利润

最小化生产成本

最小化旅行时间或距离

最大化材料强度或结构稳定性

最小化机器学习模型的预测误差

最大化投资组合回报

决策变量(Decision Variables)是模型中由决策者选择的变量，代表了决策的不同选择或方案。它们是影响目标函数和约束条件的关键因素。

例如：生产多少件产品?

每种原材料采购多少?

选择哪条路线?

设计结构的尺寸是多少?

模型的参数设为什么值?

投资多少钱到不同的 资产 ?

又例如让我决策在abc三个项目中分别投多少钱

其中x1，x2，x3为决策变量

约束条件(Constraints)是 优化模型中限制决策变量取值的条件，反映了现实中的各种限制和要求。

可用的总预算(资金限制)

原材料 的总供应量(资源限制)

工厂的最大生产能力(产能限制)

车辆的最大载重量(物理限制)

结构必须承受的最小载荷(安全要求)

法律法规要求(合规性)

变量必须是整数(离散决策)或非负数(物理意义)

那么怎么描绘这三个组成部分呢?

①线性规划模型

仅有线性项的模型

②整数规划模型

在线性的基础上加上整数的约束的模型

③0-1规划模型

只有两个选项的约束的线性模型

④非线性规划模型

有非线性项的模型

模型分类

单目标优化

多目标优化

数学规划模型的一般形式

目标函数与决策变量:

$$min/maxz(\text{目标函数}) = f(x), x = (x_1, x_2, x_3, \ldots, x_n)^T(\text{决策变量})$$

约束条件:

$$g_i(x) \leq 0, i = 1, 2, 3 \ldots, m$$

对于整数规划模型，有:

$$x \in Z$$

这个约束条件
对于0-1规划模型，有：

$$x \in \{0,1\}$$

而当f(x)、g_i(x)任意一个或两个为非线性函数时，为非线性规划模型

某奶制品加工厂生产A和B两种奶制品。其加工时长、每桶牛奶加工所得的重量、获利如下表所示：

产品	每桶牛奶加工时长	每桶牛奶加工所得的重量	获利
A	12h	3kg	24元/kg
B	8h	4kg	16元/kg

每天的可加工时长为480h，可用牛奶量为50桶，至多可加工奶制品A100kg。
求应该如何分配50桶牛奶来加工奶制品，使得总获利最大。

在这里的决策变量是：A,B，x1桶加工奶制品A，x2桶加工奶制品B

目标函数：

$$maxz = 3x_1 * 24 + 4x_2 * 16$$

约束条件：

$$\begin{aligned} x_1 + x_2 &\leq 50 \\ 12x_1 + 8x_2 &\leq 480 \\ 3x_1 &\leq 100 \\ x_1, x_2 &> 0 \\ x_1, x_2 &\in Z^+ \end{aligned}$$

要求至少选两门数学课、三门运筹学课和两门计算机课。
为了选修课程门数最少，应学习哪些课程?

课名	学分	所属类别
微积分	5	数学
线性代数	4	数学
最优化方法	4	数学；运筹学
数据结构	3	数学；计算机
应用统计	4	数学；运筹学
计算机模拟	3	计算机；运筹学
计算机编程	2	计算机
预测理论	2	运筹学
数学实验	3	运筹学；计算机

这里我们可以这样设：

$$\begin{aligned} &\text{决策变量：} x_i, \text{选第} i \text{门课程} \\ &\text{目标函数：} minZ = \sum_{i=1}^{9} x_i \\ &\text{约束条件：} x_1 + x_2 + x_3 + x_4 + x_5 \geq 2 \\ &\qquad x_3 + x_5 + x_6 + x_8 + x_9 \geq 3 \\ &\qquad x_4 + x_6 + x_7 + x_9 \geq 2 \end{aligned}$$

非线性规划模型：

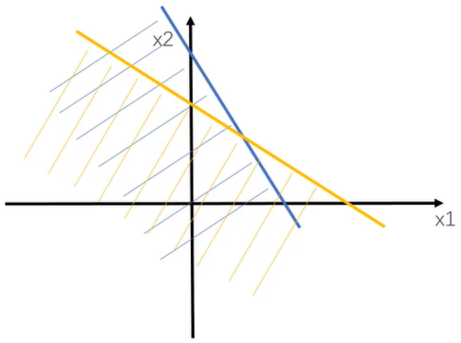
$$\begin{aligned} &b: \text{总资本} \\ &a_i: \text{第} i \text{个项目的投资额} \\ &c_i: \text{第} i \text{个项目的收益} \\ &\text{收益占总投资的比例：} \\ &maxf(x_1, x_2, x_3 \ldots, x_n) = \frac{\sum_{i=1}^{n} c_i x_i}{\sum_{i=1}^{n} a_i x_i} \\ &\text{约束条件} s.t. \begin{cases} \sum_{j=1}^{n} a_j x_j \leq b \\ x_j = 0 or 1 (j = 1, 2, \ldots n) \end{cases} \end{aligned}$$

传统求解算法的机制

设该厂生产x1台甲机床和x2台乙机床时总利润z最大，则x1 ,x2应满足：

$$maxz = 4x_1 + 3x_2$$

$$\begin{cases} 2x_1 + x_2 \le 10 \\ x_1 + x_2 \le 8 \\ x_2 \le 7 \\ x_1, x_2 \ge 0 \end{cases}$$

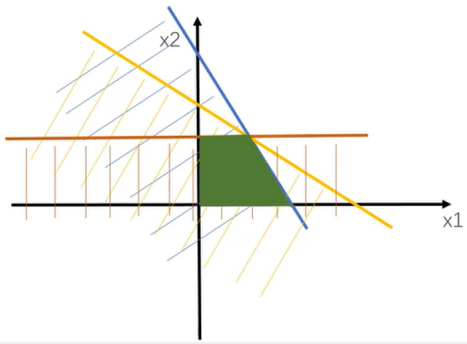


取交集法

设该厂生产x1台甲机床和x2台乙机床时总利润z最大，则x1 ,x2应满足：

$$maxz = 4x_1 + 3x_2$$

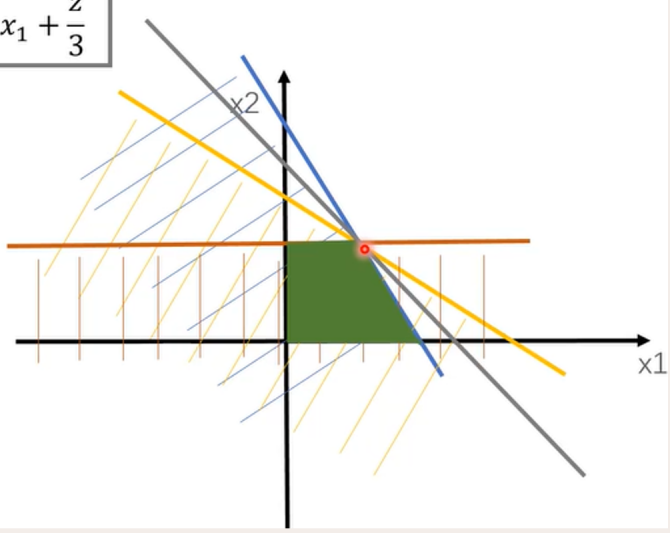
$$\begin{cases} 2x_1 + x_2 \le 10 \\ x_1 + x_2 \le 8 \\ x_2 \le 7 \\ x_1, x_2 \ge 0 \end{cases}$$



$$maxz = 4x_1 + 3x_2 \implies x_2 = -\frac{4}{3}x_1 + \frac{z}{3}$$

利润z最大，则X1 ,X2应满足：

$$x_2 = -\frac{4}{3}x_1 + \frac{z}{3}$$



讲完了传统的优化/规划模型，我们现在要了解主流的算法——智能优化算法

对于上面我们讲的传统求解算法的机制，我们用的就是求精确解的方法

求精确解是严格按照数学推断来进行的

相对的，求近似解就需要我们不断迭代得到一个效果较好的结果

求精确解的求解器

python接入Gurobi

优化问题	Python中常用模块	实际求解器	主要用途
连续线性规划	`scipy.optimize.linprog`	HiGHS	决策变量允许取小
混合整数线性规划 MILP	`scipy.optimize.milp`	HiGHS	部分或全部变量为整数、0-1变量
大规模 LP/MIP	`gurobipy`	Gurobi	排班、选址、路径、生产计划等
大规模 LP/MIP	`docplex.mp` 或 `cplex`	CPLEX	与 Gurobi 类似
通用建模接口	`Pyomo`、`PuLP`	需要另接 HiGHS、CBC、Gurobi、CPLEX 等	负责写模型，本身通常不是求解器

精确解求法

- 数学规划
  - 单纯形法
  - 内点法
- 分支定界法
- 动态规划
- 分治法
- 枚举法

近似解求法

启发式算法（就是 按照一个规则去求解，按规则来解，就不会怎么生成低效解）

- 例如家庭小菜馆，上菜原则一般是谁先点就先做谁的菜
- 但是大饭馆可能就是越多人点，那么干脆晚一点一锅子炒完
- 贪心算法
- 局部搜索算法
- 启发式求解的方式
  - 1.时间窗抢占规则
    优先去关闭时间最早的景点(如夜景天台)，**避免错过关键体验**。
  - 2.道具依赖链规则
    强制将依赖道具的活动(奶茶店拍照)排在道具获取点(买花)之后，且中间无禁带场所。
  - 3.近邻贪婪规则
    从当前位置选择距离最近且符合约束的下一个点(如影院散场后优先去附近天台而非返程买花)。

元启发式算法（不是单指一个算法，而是 **一种元启发式算法其实泛指一类算法**）

- 遗传算法GA
- 模拟退火算法SA
- 蚁群算法ACO
- 粒子群算法PSO
- 禁忌搜索TS
- 核心思想:为了解决复杂问题，需要一种更高级的策略来管理搜索过程，**平衡对解空间的探索(寻找新区域)和勘探(在好区域深度挖掘)**，并避免过早陷入局部最优解。

智能优化算法的分类

- 单点优化算法
  - 模拟退火算法
- 群智能算法

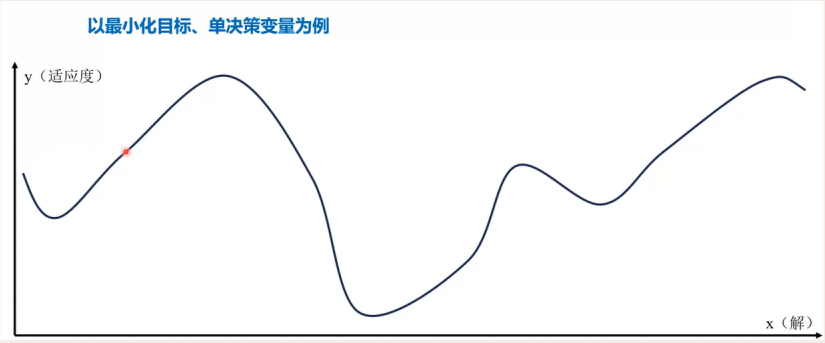
这两者的区别可以用一个例子来解释：前者是单人扫雷；后者是多人扫雷

遗传算法、粒子群算法

单目标优化

多目标优化

智能优化算法的求解思路



以最小化目标，只有一个决策变量的例子来说

例如采购方案为x，那么对应的需要的成本为y

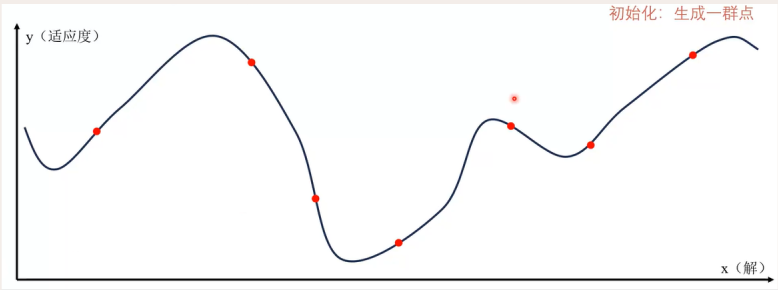
那么我怎么知道这个采购方案可以最小化成本呢？要么我已经知道；要么我一个个试出来

我可以选择一个个试出来，但是问题在于一方面成本大，另外一方面可能解不能接受

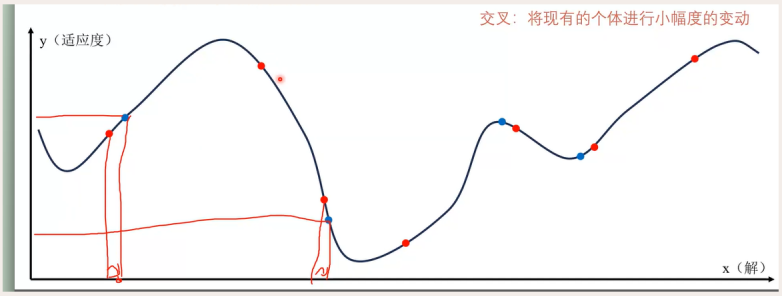
例如我要采购苹果，采购4.5筐苹果最划算，但是怎么把筐一分为二呢？

那么在这里，我们可以这样做

①初始化，随机找一些点试一试

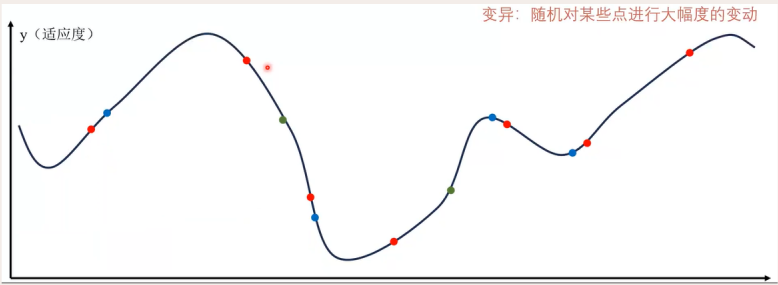


②交叉，在点的附近探索一下，趋势是递增还是递减？



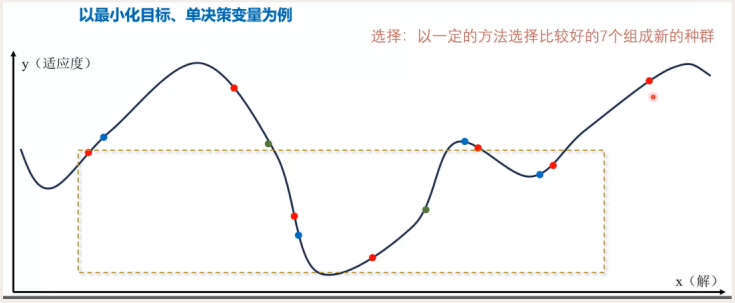
这种变动是小幅度的，随机方向的变动

③变异，随机选某些点进行大变动



④选择，选择点的方式造成了智能优化算法下面子算法的分野

贪婪算法，就选择了上面三步的最好的几个点



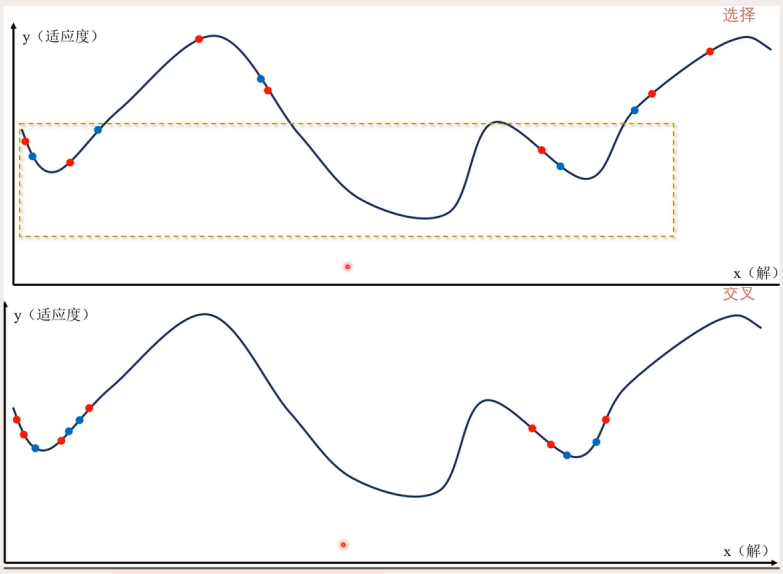
之后重复上面三步骤，然后再选里面最好的几个点，然后再重复

这就是**找新区域，选出目前最好的区域后深度挖掘**

现在我们向其他一种方案看

如果我去掉步骤三变异

于是我在第二步之后直接到选择环节，选择完之后再小变动，之后继续直接开始选点



然后就发现**你被困在局部最优的区域了**

现在我们**只要变异，而不是小变动（即交叉），那么就可能越训练越回去了**

因为大变动可以理解为一种探索，而不是收敛于一个区域

这就是探索和勘探的平衡

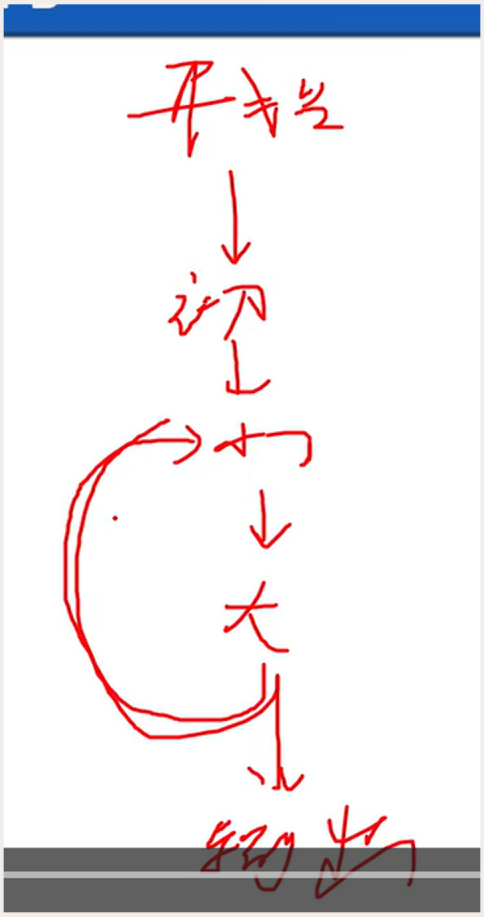
还有一些术语需要注意，例如最终的解不能称为"最优解"，只能称为"近优解"

除了贪婪算法之外，我们还可以在上面所说的"选择"这一步里面使用其他的策略

选入概率大的，更容易进入下一次迭代；被选入概率小的，迭代过程中也有机会

进入下一次迭代

改进方向



在这一个初始过程里面，我们可以在大变动做完之后我加一个算子，用一些知识来引导搜索，让搜索更高效

以遗传算法为例子来分析一下

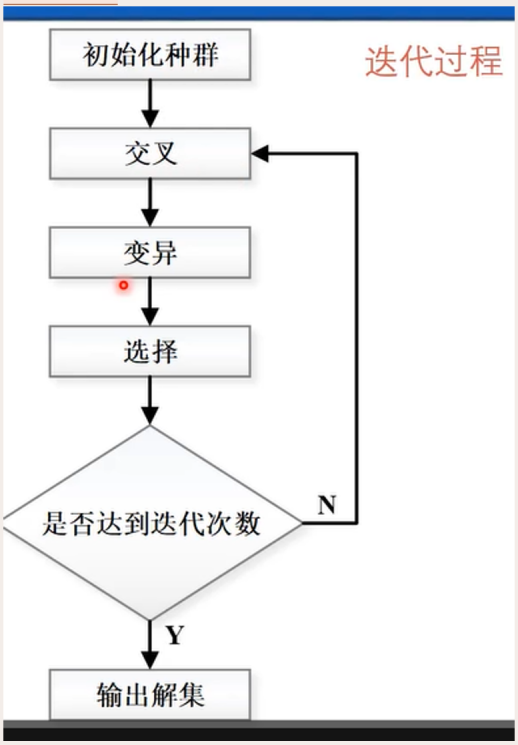
前面已经说明，优化问题需要先确定决策变量、目标函数和约束条件。完成这一步以后，我们得到的是一个数学模型，但模型本身并不会自动给出答案，还需要选择具体的求解算法。

数学模型回答的是： **什么样的方案可行，以及怎样比较两个方案的优劣。**

求解算法回答的是： **怎样在大量可行方案中找到目标函数值较好的方案。**

以旅行商问题为例，0-1规划模型可以用变量 x_{uv} 表示是否从 **城市 (u) 直接前往城**

市 (v)；遗传算法则不直接操作这个0-1矩阵，**而是把一条完整的城市访问顺序编码成"染色体"**。两种表示描述的是同一个问题，但承担的任务不同。



遗传算法的过程如上，就是初始化=》交叉=》变异=》选择=》迭代=》交叉...

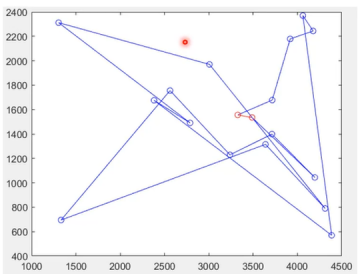
旅行商问题（TSP问题）

假设有一个旅行商人要拜访全国31个省会城市，他需要选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。

对路径选择的要求是：最小化总路程长度

已知城市坐标为：

```
1304 2312;    4386 570;
3639 1315;    3007 1970;
4177 2244;    2562 1756;
3712 1399;    2788 1491;
3488 1535;    2381 1676;
3326 1556;    1332 695;
3238 1229;    3715 1678;
4196 1044;    3918 2179;
4312 790;     4061 2370;
```



旅行商问题(TSP问题)

假设有一个旅行商人要拜访全国31个省会城市，他需要选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。对路径选择的要求是:最小化总路程长度

在这个问题中，其中最佳的一种建模是：

$$min \sum_{u,v} x_{(u,v)} c_{(u,v)}$$

其中$x_{(u,v)}$是关于$0-1$变量的矩阵

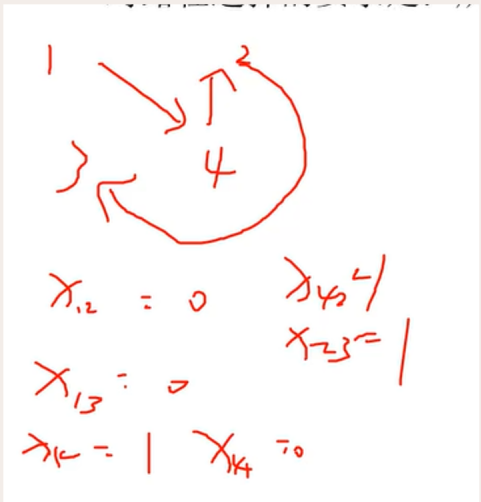
u代表出发的城市，v代表到达的城市，若u和v之间有一个有向线段的路径，那么就取1

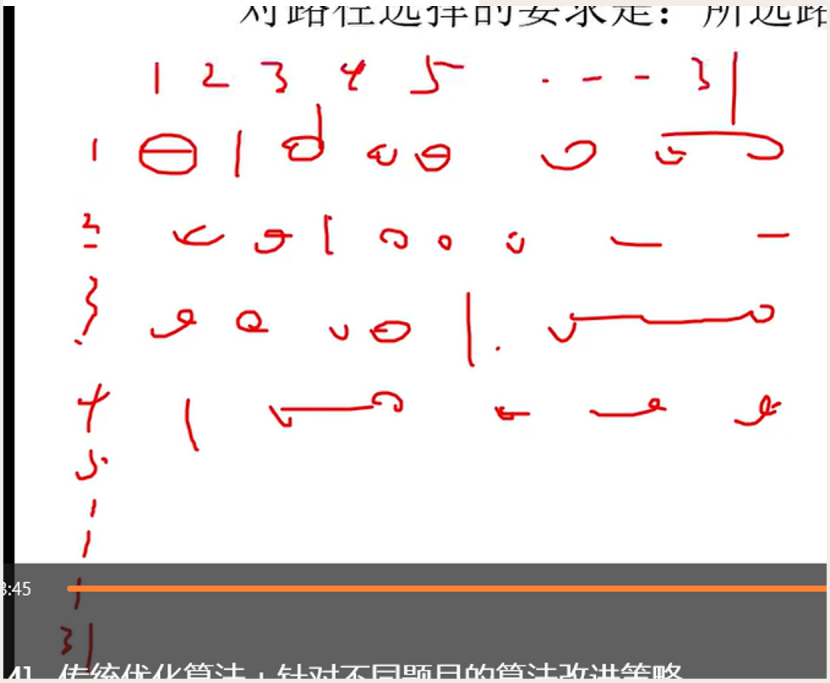
否则取0

$c_{(u,v)}$是路程

前文已经用0-1变量建立了TSP模型，包括：每个城市只能有一条出发路径、每个

城市只能有一条进入路径，以及不能形成多个相互独立的子回路。这里不再重复完整模型，而是讨论遗传算法如何搜索路线。





如上图，一个是用实际路径表示，一个是用矩阵表示

接下来翻译约束条件：首先每个城市只能拜访一次，那么规定：

$$\sum_{u} x_{(u,v)} = 1, \forall v \in V,$$

就例如根据这个矩阵来说，行层面的规则是：只能有一条路

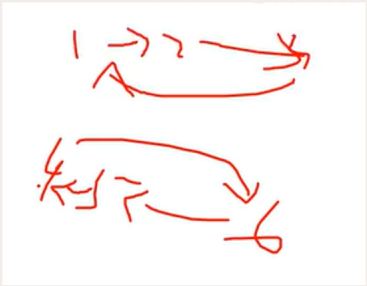
例如城市1,2,3；到城市1的路径只能有2-》1,和3=>1

还有：

$$\sum_{v} x_{(u,v)} = 1, \forall v \in V$$

即城市1出发，只能去一个城市而不是既去城市2又去城市3

从矩阵来理解的话，就是列层面上要求“闭环”



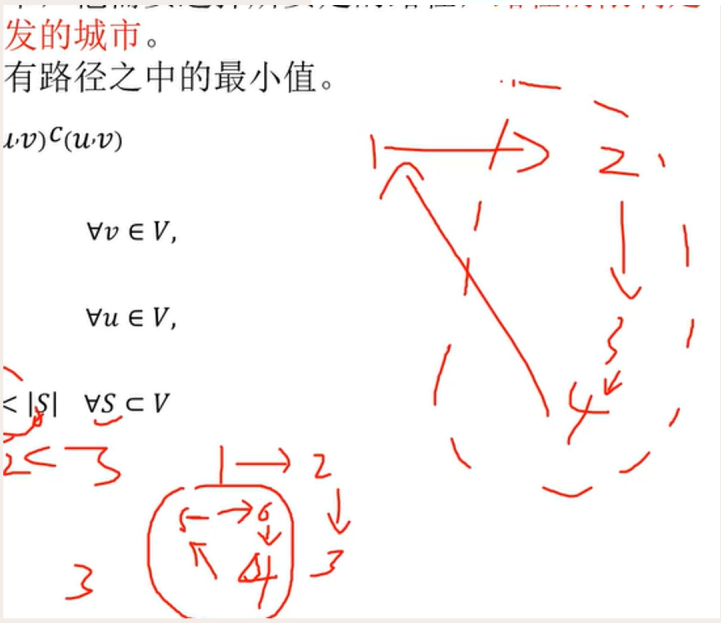
那么最后一个约束条件，约束的是这种情况：

因为人不能从1=》4直接瞬移：

$$\sum_{u\in S}\sum_{v\in S} x_{(u,v)} < |S|, \forall S \subset V$$



其中|S|代表有完整路径的子环，



例如上面1=》2=》3=》4=》1，S代表我走过多少城市，2（2条有效路径）<3(3个城市)

下面的1=>2=>3；5=>6=>4=>5,这就3<3不成立了

遗传算法怎么做?

遗传算法首先要把一个可行方案转化为染色体。

对于该问题，可以用城市编号的排列表示访问顺序。例如：

$$\pi = (4, 6, 9, 2, 3, 8, 7, 11, 10, 1)$$

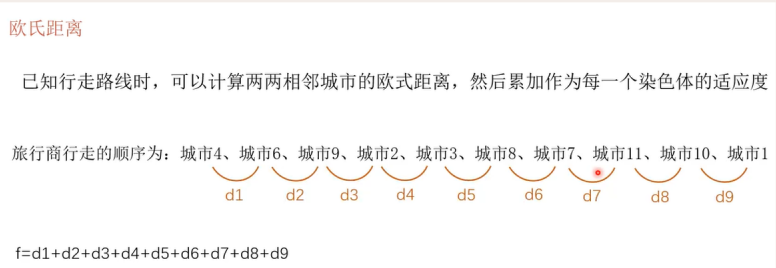
表示旅行商依次访问城市4、城市6、城市9.....最后访问城市1，并从城市1返回城市4。

这种编码有一个直接好处： **只要序列中每个城市恰好出现一次，它就天然满足“每个城市只能访问一次”的要求，也不会出现两个彼此分离的子回路。**

因此，遗传算法不需要在每次迭代时重新检查完整的0-1规划约束，只需要保证交叉和变异之后的序列仍然是城市编号的合法排列。（这个也是我们要在数学建模里面需要思考的，什么优化算法适用于这个问题）

已知行走路线时，可以计算两两相邻城市的欧式距离，然后累加作为每一个染色体的适应度

旅行商行走的顺序为:城市4、城市6、城市9、城市2、城市3、城市8、城市7、城市11、城市10、城市1



例如：设染色体为：

$$\pi = (\pi_1, \pi_2, \ldots, \pi_n)$$

对应的总路程为：

$$\sum_{k=1}^{n-1} c_{\pi_k, \pi_{k+1}} + c_{\pi_n, \pi_1}$$

前一项计算相邻城市之间的路程，最后一项计算最后一个城市返回起点的路程。

旅行商问题的目标是最小化 L(\pi)。但遗传算法通常规定适应度越大，个体越好，因此可以将适应度写成：

$$F(\pi) = \frac{1}{L(\pi) + \varepsilon}$$

其中，$\varepsilon > 0$用于防止分母为0。

路线越短，$L(\pi)$越小，对应的适应度$F(\pi)$越大。

之后的步骤是初始化种群

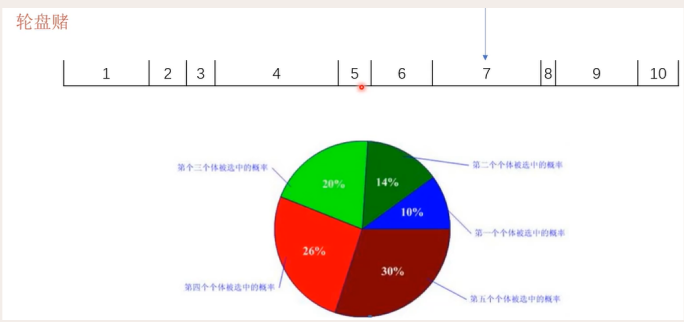
遗传算法不会只生成一条路线，而是同时生成多条候选路线，这些候选路线构成初始种群。例如，种群规模设为100，就随机生成100个城市排列：

$$P_0 = \pi^{(1)}, \pi^{(2)}, \ldots, \pi^{(100)}$$

完全随机初始化能够增加路线的差异，但也可能生成大量质量很差的路线。改进时可以让一部分路线随机生成，另一部分路线通过最近邻等规则生成，使初始种群同时保留随机性和一定的路线质量。

之后是选择

我们可以用这样的选择方式



越好的点，被选中的概率越大；不好的点选中的概率越小

计算每条染色体的适应度以后，需要从当前种群中选择父代。选择的基本原

则是：**适应度越高，进入繁殖环节的概率越大**

但这并不意味着只保留当前最好的若干条路线。

如果每次都机械地选择最优个体，种群会很快变得相似，后续产生的新路线也会越来越接近，容易停留在局部较优区域。

因此，可以使用轮盘赌选择法/锦标赛选择法。

较好的路线更容易被选中，较差路线仍保留较小概率进入下一环节，以维持种群差异。

例如

> 小变动的方法可以不一样，那么这里是随机选三个连续的点：10,7,8
> 那么另外一个解对应看这三个数字在哪，如图是8,7,10顺序
> 那么我们把8,7,10放在第一个解，10,7,8放在第二个解，那么顺序就换了

实际计算中还可以使用精英保留：把当前最好的少量路线直接复制到下一代，防止交叉或变异将已经找到的较好解破坏。



不同问题里面需要不同的小变动（即“交叉”）

交叉的作用是**把两条父代路线中的部分结构重新组合，得到新的子代路线。**

旅行商问题采用排列编码，不能直接使用普通二进制交叉。

普通交叉可能造成某些城市重复出现，另一些城市完全消失。例如，子代可能出现两个城市7，却没有城市9，这样的路线不再可行。

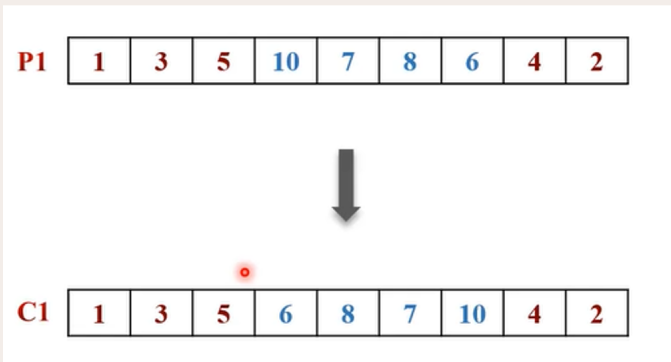
因此，需要使用适合排列编码的交叉方法，例如顺序交叉或部分映射交叉。

其基本做法是：先从父代A中保留一段连续城市，再按照父代B中的相对顺序补入剩余城市。这样得到的子代仍然包含全部城市，并且每个城市只出现一次。

交叉主要利用已经发现的较好路线片段。

例如父代A中的“城市4—城市6—城市9”距离较短，父代B中的“城市2—城市3—城市8”结构较好，交叉可以尝试把这两个局部结构组合到同一条新路线中。

变异怎么做：



这里就是我随机选4个连续数字，然后逆序

变异是在一条路线内部做随机调整，以产生当前种群中没有出现过的新结构。

旅行商问题常用的变异方式包括交换两个城市的位置，或者随机选择一段城市并将其逆序。上图中的做法属于逆序变异

再比如：

$$\begin{aligned} \text{例如：}&(4,6,9,2,3,8,7,11,10,1)\\ \text{选中连续片段：}&(2,3,8,7)\\ \text{将其逆序后得到：}&(4,6,9,7,8,3,2,11,10,1) \end{aligned}$$

逆序操作不会增加或删除城市，因此变异后的路线仍然可行。

这样变化，都是为了不能违反约束

生成下一代与终止条件

完成选择、交叉和变异以后，可以得到新的候选路线。将子代与保留的精英个体组合，形成下一代种群：

$$P_t \longrightarrow P_{t+1}$$

之后重新计算路径长度和适应度，再进行下一轮选择、交叉和变异。

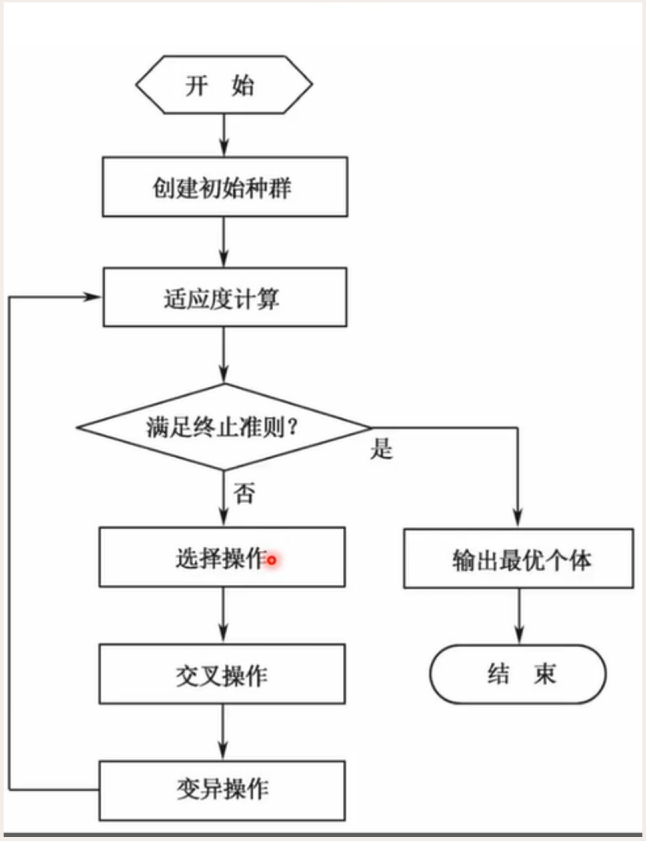
常见的终止条件有两类：

一是达到预先设定的最大迭代次数；

二是连续若干代没有找到更短的路线。算法终止后，输出迭代过程中找到的最短路线。

遗传算法没有遍历全部路线，也没有通过上下界证明当前路线一定是全局最优，因此结果应称为：当前最优解、近优解或较优可行解

整体流程图：



对于遗传算法GA的评论

遗传算法需要同时处理两件事：一方面继续寻找新的路线结构（探索），另一方面在已经发现的较好路线附近继续改进。（挖掘）

选择和交叉会让种群逐渐向较好的路线集中。它们主要利用当前已经发现的信息，使算法逐步收敛。

变异会引入新的城市排列，为算法提供跳出当前区域的机会。如果完全取消变异，种群中的路线会越来越相似，算法可能停留在某个局部较优区域。

变异概率也不能设置得过高。变异过于频繁时，上一代积累的较好路线结构会不断被破坏，算法会接近随机搜索，很难稳定收敛。

多目标优化

如果说前面的TSP只有一个目标：min L(\pi)

因此，每条路线都可以用一个总路程数值进行比较。

但现实问题可能同时考虑路程、费用、风险和满意度。例如，某条路线距离最短，但收费较高；另一条路线费用较低，但耗时更长。此时无法仅用一个原始目标判断哪条路线一定更好。

多目标优化常见的处理思路有三种。

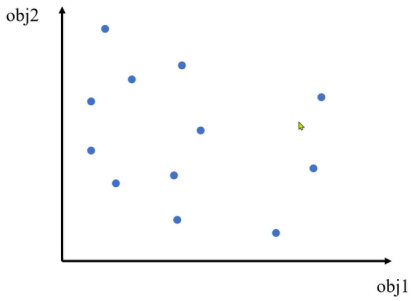
思路①谁比谁更重要，那么优先达到谁？

先确定目标之间的优先顺序。

例如 **先最小化风险，在风险达到可接受水平的方案中，再最小化成本；最后再比较时间。**

这种方法 **适合目标优先关系比较明确的情形，但排序结果会受到优先级设定影响。**

思路1：次序优化　　假设obj1远比obj2重要



详细地就是：先找到我最想达到的目标的最好的几个解；最后再从中筛选

思路②加权优化，就是把重要性变成权重的形式

权重1* 变量1+权重2* 变量2 得到总评分，选择总评分高的

设有 (M) 个目标函数，可以写成：

$$\min F(x) = \sum_{m=1}^{M} w_m f_m(x)$$

其中：

$$w_m \geq 0, \qquad \sum_{m=1}^{M} w_m = 1$$

权重表示不同目标的重要程度。加权后，多目标问题重新转化为单目标问题，可以继续使用普通遗传算法求解。

这种做法计算方便，但权重通常需要提前确定。权重稍有变化，最终选择的方案也可能发生变化。

思路③帕累托优化（重点）

帕累托优化不要求提前把多个目标压缩成一个总分，而是保留多种各有取舍的方案。

假设所有目标都需要最小化。对两个可行解 (a) 和 (b)，如果：

$$f_m(a) \leq f_m(b), \qquad \forall m = 1, 2, \ldots, M$$

并且至少存在一个目标满足：

$$f_m(a) < f_m(b)$$

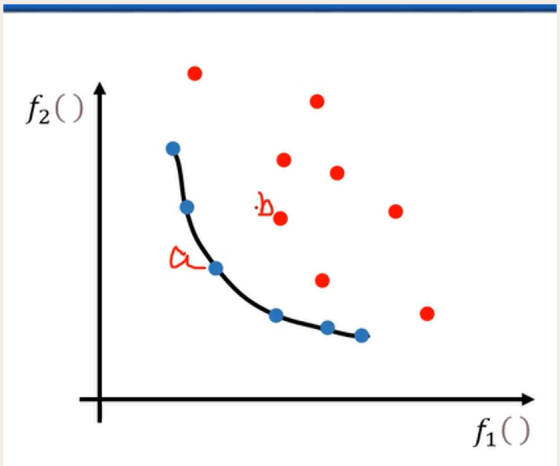
,则称 **解 (a) 支配解 (b)** 。

这表示 (a) 在所有目标上都不比 (b) 差，并且至少有一个目标严格优于 (b)。在这种情况下，(b)没有继续保留的必要。

如果不存在其他可行解支配 (a)，则称 (a) 为非支配解或帕累托解。所有非支配解组成的集合称为帕累托前沿。

帕累托前沿上的方案无法继续进行绝对比较。

某个方案可能成本更低但耗时更长，另一个方案可能耗时更短但成本更高。模型先给出这些折中方案，再由决策者根据实际偏好选择其中一个。



如图，我的目标是最小化，那么a在两个目标上全面优于b，那么b会被抛弃，a会被选择

在黑线上，尽管有些点的表现不尽如人意，但是各有所长

于是这就是帕累托前沿

那么遗传算法经过帕累托选择之后，就变成NSGA-II

你可以将其理解为：遗传算法的多目标扩展

NSGA-II保留了遗传算法的编码、交叉和变异，同时改变了个体评价与种群更新方式。

普通单目标遗传算法（GA）主要根据适应度大小选择个体；NSGA-II使用两个判断标准：

①非支配等级越靠前，个体越优先；

②处于同一非支配等级时，拥挤距离越大，个体越优先。

非支配排序

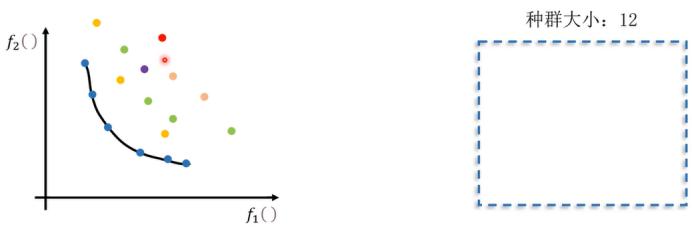
把当前种群中的个体分成若干层：

$$F_1, F_2, F_3, \dots$$

其中，(F_1)是当前种群中不被任何其他个体支配的解；去掉 (F_1) 后，剩余解中的非支配解组成 (F_2)；继续重复，得到后续各层。

非支配排序遗传算法（NSGA-Ⅱ）的选择步骤

从帕累托前沿开始一层一层往后选



我们要从中选出12个解，但是最前沿只有6个解，但没关系，选完这个我往后筛一层直到满足为止

但是如果前沿的个数>我选取的个数呢？我就找最多样的

拥挤距离

如果某一层中的解数量超过剩余名额，只看非支配等级就无法继续区分，因为这些解位于同一层。

此时需要优先保留分布较为分散的解，使得到的帕累托前沿尽量覆盖不同的权衡区域，而不是全部集中在某一小段。

拥挤距离的计算

在某一个非支配层内，对每个目标函数分别进行升序排列。

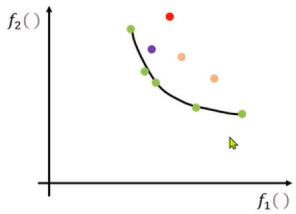
对于第 (m) 个目标，排在最前端和最后端的两个边界个体设为：

$$d_1 = d_L = \infty$$

这样可以保留帕累托前沿两端的边界方案。

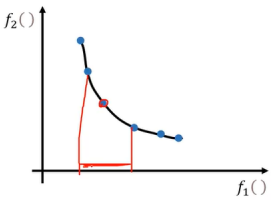
对于非边界个体 (i)，拥挤距离为：

$\sum_{m=1}^{M} \frac{f_m^{(i+1)} - f_m^{(i-1)}}{f_m^{\max} - f_m^{\min}}$，其中，$f_m^{(i-1)}$和$f_m^{(i+1)}$是个体$i$在第$m$个目标上相邻两个个体的目标函数值。





拥挤距离



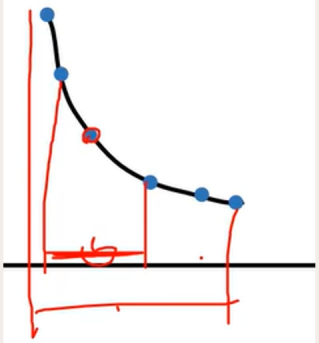
NSGA-II算法通过计算拥挤距离来维持种群的多样性。拥挤距离描述群体个体所处环境的拥挤程度（密度）和目标函数值有关。计算方法如下[1]：

1．设个体的拥挤距离用d表示，个体i的拥挤距离用d表示，设置$d_i = 0$

2．设f_m为目标函数，$m = 1,2,..M$基于每个目标函数值对所有个体进行升序排列，将排在第一位即函数值最小值的个体的拥挤距离和排在最后一位即函数值最大值的个体的拥挤距离设为$d_1 = d_L = \infty$,即对每一目标函数，边界个体（具有最大和最小函数值的个体）对应的拥挤距离值设置为无穷大。

3．非边界个体i拥挤距离度量值的计算方法为：

$$d_i = \sum_{m=1}^{M} \frac{|f_m(i-1) - f(i+1)|}{f_m^{max} - f_m^{min}}$$



分子表示相邻两个解之间的跨度，

分母表示该目标在当前非支配层中的总取值范围。

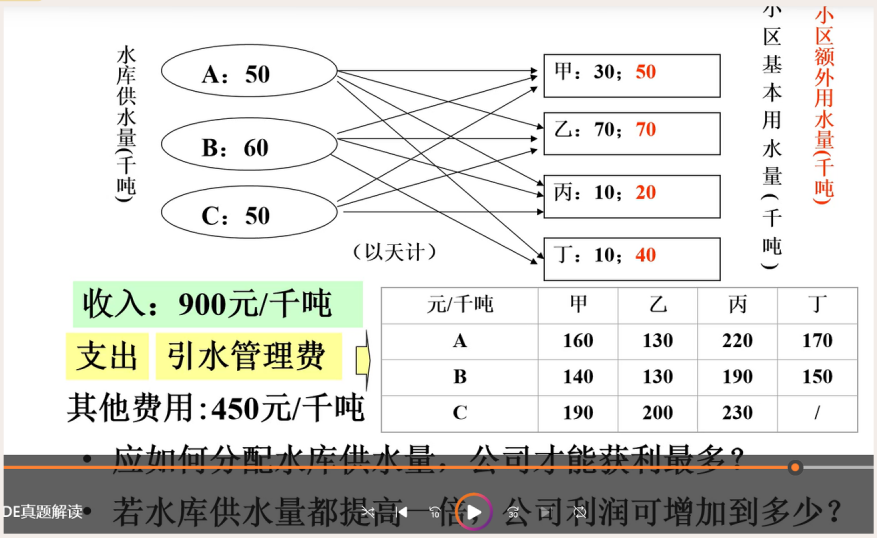
经过分母归一化后，不同量纲的目标可以共同参与距离计算。

拥挤距离越大，说明该个体周围的其他解越少，所处区域越稀疏；

拥挤距离越小，说明该区域已经存在较多相似解。

因此，当某一非支配层不能全部保留时，应优先选择拥挤距离较大的个体。

优化模型的变量设置

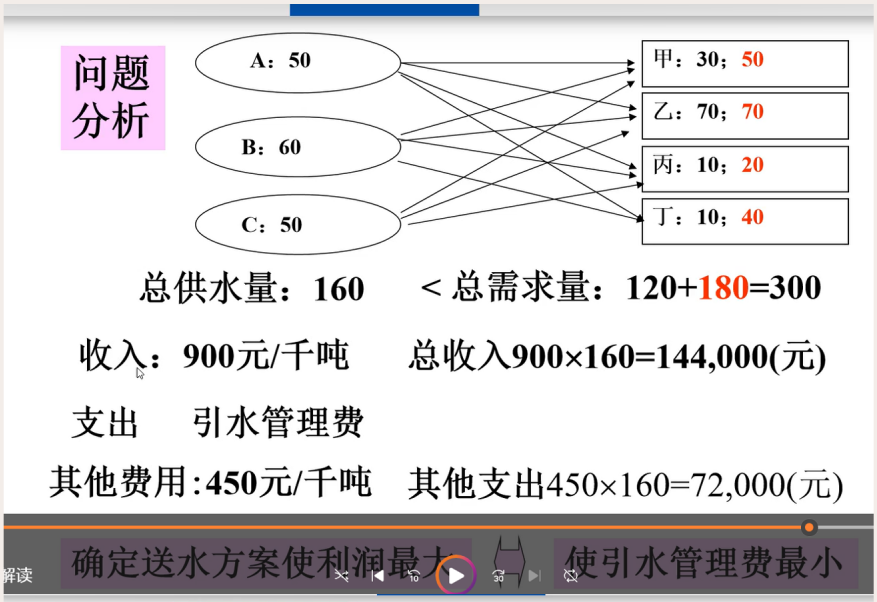


这里所讲的变量设置，就是设x，设自变量的方法

例如这一道题，有4×3种分配方案，你当然可以一个个方案作为自变量，但是你理解起来就很困难，计算的时候就很难受，工作量也大

于是，我们不妨分析一下问题，在这个水库中，我们不妨设水库i向j区的日供水量为：xij

（其中x_{34}为0）



模型建立　确定3个水库向4个小区的供水量

决策变量　水库i 向j 区的日供水量为 x_{ij}（x_{34}=0）

目标函数

$$Min \quad Z=160x_{11}+130x_{12}+220x_{13}+170x_{14}$$
$$+140x_{21}+130x_{22}+190x_{23}+150x_{24}+190x_{31}+200x_{32}+230x_{33}$$

约束条件

供应限制

$$x_{11}+x_{12}+x_{13}+x_{14}=50$$
$$x_{21}+x_{22}+x_{23}+x_{24}=60$$
$$x_{31}+x_{32}+x_{33}=50$$

需求限制

$$30 \le x_{11}+x_{21}+x_{31} \le 80$$
$$70 \le x_{12}+x_{22}+x_{32} \le 140$$
$$10 \le x_{13}+x_{23}+x_{33} \le 30$$
$$10 \le x_{14}+x_{24} \le 50$$

线性规划模型(LP)

这就是运输问题

给定多个供应点和需求点的情况下，如何通过运输路线将物资从供应点运到需求点，以达到运输成本最小的目标

每个供应点有固定的供应量，每个需求点有固定的需求量，而每条路线的运输成本不同

目标：总运输成本最小，同时满足供应和需求的约束条件

决策变量

在运输问题中，通常定义一个决策变量xij表示来表示 **从供应点i向需求点j运输的数量。** 该变量是连续的，并且具有非负性约束，即xij≥0

目标函数

目标是最小化运输总成本

假设 **cij表示从供应点i到需求点j的单位运输成本** ，**xij表示从供应点i到需求点j的运输量** 。那么总成本Z可以表示为:

$$\min Z=\sum_{i=1}^{m}\sum_{j=1}^{n}c_{ij}x_{ij}$$

其中，m为供应点的数量，n为需求点的数量。

约束条件

供应约束:每个供应点的供应量不能超过其总 供给 。

对于每个供应点i:

$$\sum_{j=1}^{n} x_{ij} = s_i, \forall i = 1, 2 \dots m$$

其中，si表示供应点i的供给量。

需求 约束:每个需求点的需求量必须满足。

对于每个需求点j:

$$\sum_{i=1}^{m} x_{ij} = d_j, \forall j = 1, 2, \dots, n$$

其中，dj表示需求点j的需求量。

非负约束：

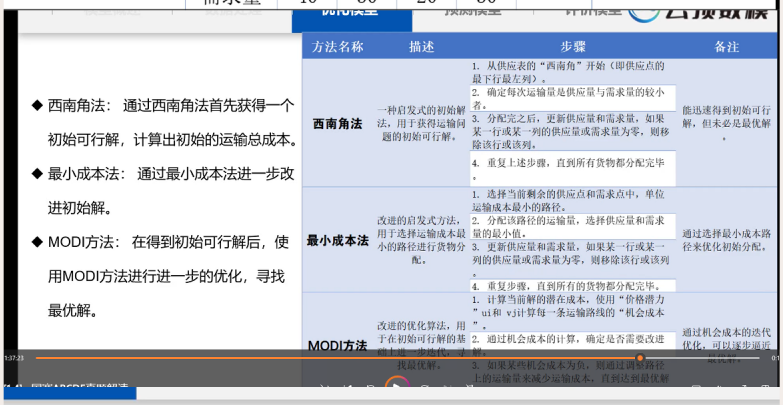
$$x_{ij} \geq 0, \forall i, j$$

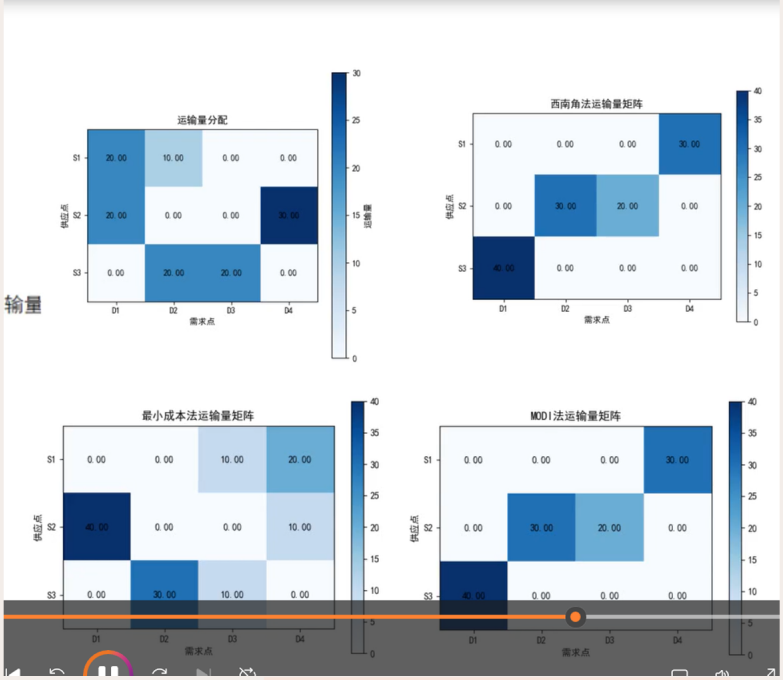
再来一个案例

实际案例展示与解析

假设有一个小型的运输问题，涉及3个供应点和4个需求点。供应点和需求点的供给量和需求量，以及每条运输路线的单位运输成本如下表所示：

供应点/需求点	D1	D2	D3	D4	供给量
S1	4	6	8	10	30
S2	2	5	7	6	50
S3	3	4	5	7	40
需求量	40	30	20	30	





你会发现这里有三种方法，**三种方法的结果都不一样，这是因为是求近似解**

指派问题

目标：将一组任务指派给一组工人/一组资源，使完成的总成本最小

决策变量

设xij为决策变量，表示工人i是否执行任务j。

其中，xij=1表示工人i执行任务j，xij=0表示工人i不执行任务j

目标函数

总成本:最小化所有任务分配的总成本，即:

$$\min Z = \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

其中，cij是工人i执行任务j的成本。

约束条件

每个工人只执行一个任务:

$$\sum_{j=1}^{n} x_{ij} = 1, \forall i = 1, 2, \ldots, n$$

每个任务只能由一个工人执行:

$$\sum_{i=1}^{n} x_{ij} = 1, \forall j = 1, 2, \ldots, n$$

变量取值为0或1:

$$x_{ij} \in (0, 1), \forall i, j$$

旅行商问题

一名销售员想要访问若干城市，每个城市有且仅访问一次，并且最终需要返回起始城市

目标是最小化销售员的总旅行距离/总费用

决策变量

令xij为一个二元决策变量，表示是否从城市i直接前往城市j:

$$x_{ij} = \begin{cases} 1, 如果从城市i到城市j有路径 \\ 0, 如果从i到j没有路 \end{cases}$$

目标函数

最小化旅行商的总路径长度，目标函数为:

$$\min Z = \sum_{i=1}^{n} \sum_{j=1, j \neq i}^{n} c_{ij} x_{ij}$$

其中，cij表示从城市i到城市j的距离。

约束条件

每个城市仅被访问一次:从城市i只能有一条出发路径:

$$\sum_{j \neq i} x_{ij} = 1, \forall i$$

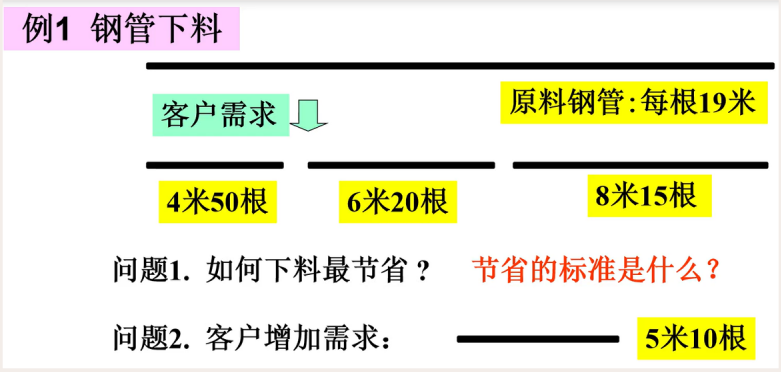
同理，城市j只能有一条进入路径:

$$\sum_{i \neq j} x_{ij} = 1, \forall j$$

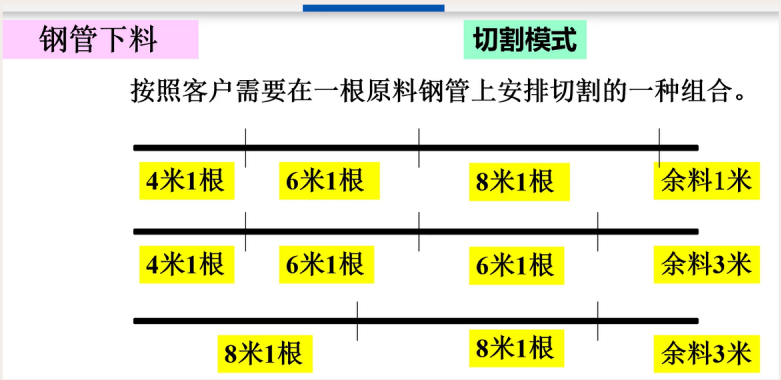
去除子回路(用于去除路径中的子回路):对于任意的城市集合S ⊂ {1,2,...,n},且2≤|S|≤n-1,我们要避免形成子回路，约束条件为:

$$\sum_{i \in S} \sum_{j \in S, i \neq j} x_{ij} \leq |S| - 1$$

例如这里的钢管下料问题



问什么设什么，下料最节省=》要裁多少次



那么我们需要先知道有多少种裁剪方案

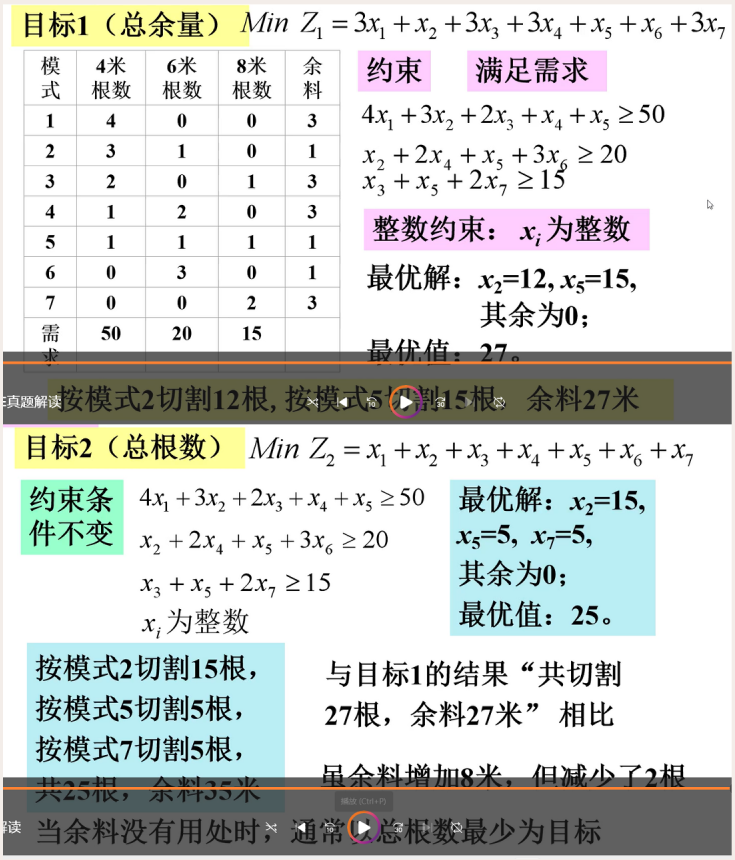
模式	4米钢管根数	6米钢管根数	8米钢管根数	余料(米)
1	4	0	0	3
2	3	1	0	1
3	2	0	1	3
4	1	2	0	3
5	1	1	1	1
6	0	3	0	1
7	0	0	2	3

为满足客户需要，按照哪些种合理模式，每种模式切割多少根原料钢管，最为节省？

我们最终得到了7个方案

决策变量

xi =》按照第i种模式切割的原料钢管根数(i=1,2,3...7)



以下是关于旅行商问题的解析：旅行商问题(TSP问题)

假设有一个旅行商人要拜访全国31个省会城市，他需要选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。对路径选择的要求是：最小化总路程长度

在这个问题中，其中最佳的一种建模是：

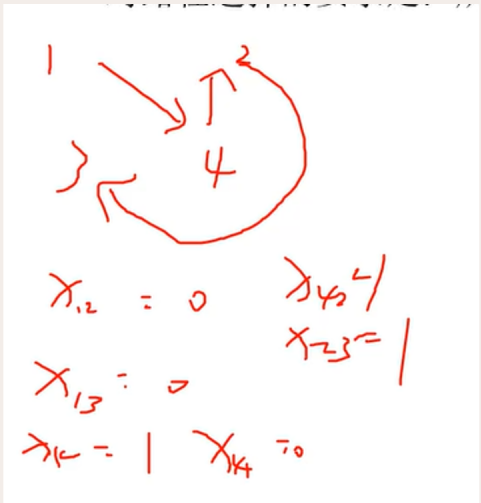
$$min \sum_{u,v} x_{(u,v)} c_{(u,v)}$$

其中$x_{(u,v)}$是关于$0-1$变量的矩阵

u代表出发的城市，v代表到达的城市，若u和v之间有一个有向线段的路径，那么就取1

否则取0

$c_{(u,v)}$是路程



如上图

接下来翻译约束条件：首先每个城市只能拜访一次，那么规定：

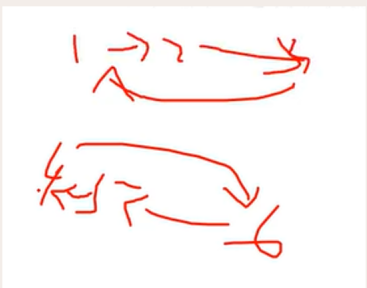
$$\sum_{u} x_{(u,v)} = 1, \forall v \in V,$$

例如城市1,2,3；到城市1的路径只能有2-》1,和3=>1

还有：

$$\sum_{v} x_{(u,v)} = 1, \forall v \in V$$

即城市1出发，只能去一个城市而不是既去城市2又去城市3

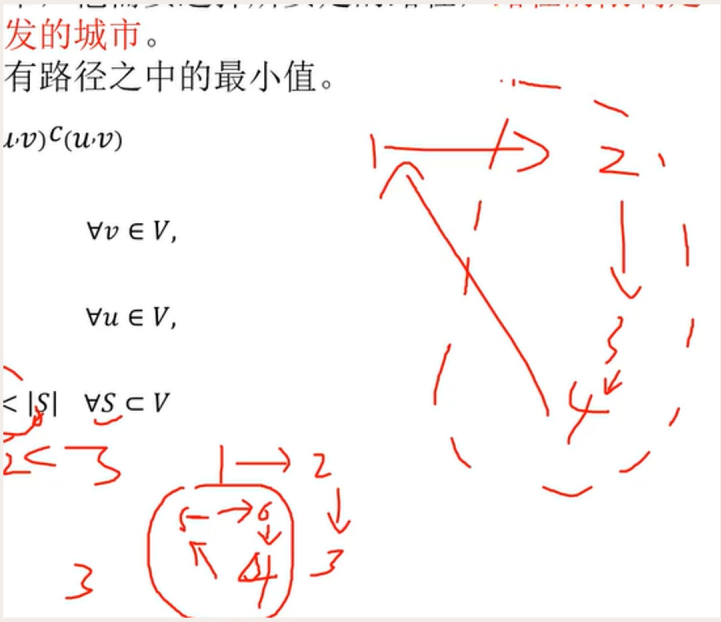


那么最后一个约束条件，约束的是这种情况：

因为人不能从1=》4直接瞬移：

$$\sum_{u \in S} \sum_{v \in S} x_{(u,v)} < |S|, \forall S \subset V$$

其中|S|代表有完整路径的子环，



例如上面1=》2=》3=》4=》1，S代表我走过多少城市，2（2条有效路径）<3(3个城市)

下面的1=>2=>3；5=>6=>4=>5,这就3<3不成立了

链接当前文件 2

优化问题的模型介绍 2

[[数据分析方法——优化模型求解]]

[[数据分析方法——优化模型求解]]

提到当前文件名